

\Programación de interacción

Objetivo: En esta actividad se mostrarán evidencias de los bloques de códigos principales de las interacciones del sistema.

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <title>Detector de Billetes USD</title>

  <style>
    html,
    body {
      margin: 0;
      padding: 0;
      height: 100%;
      background: black;
      overflow: hidden;
      font-family: Arial, sans-serif;
    }

    #camera {
      position: fixed;
      top: 0;
      left: 0;
      width: 100%;
      height: 100%;
      object-fit: cover;
      z-index: 1;
    }

    #mainButton {
      position: fixed;
      bottom: 35px;
      left: 50%;
      transform: translateX(-50%);
      width: 70%;
      max-width: 450px;
      background: #3866FF;
      padding: 30px 20px;
      border-radius: 25px;
      border: none;
      color: white;
      text-align: center;
      z-index: 3;
      cursor: pointer;
      box-shadow: 0px 8px 20px rgba(0, 0, 0, 0.4);
    }

    #mainButtonText {
      font-size: 32px;
      font-weight: 900;
      margin: 0;
    }

    #subText {
      font-size: 18px;
      opacity: 0.8;
      margin-top: 8px;
    }

    /* Resultado */
    #resultBox {
      position: fixed;
      bottom: 35px;
      left: 50%;
      transform: translateX(-50%);
      width: 70%;
      max-width: 450px;
      background: #3866FF;
      padding: 40px 20px;
      border-radius: 25px;
      display: none;
      z-index: 4;
      cursor: pointer;
      text-align: center;
      box-shadow: 0px 8px 25px rgba(0, 0, 0, 0.45);
    }

    #resultText {
      font-size: 34px;
      font-weight: 900;
      margin-bottom: 10px;
    }

    #totalText {
      position: fixed;
      top: 20px;
      left: 50%;
      transform: translateX(-50%);
      font-size: 26px;
      font-weight: bold;
      color: white;
      text-shadow: 0 0 5px black;
      z-index: 5;
    }

    #totalBox {
      position: fixed;
      top: 20px;
      left: 50%;
      transform: translateX(-50%);
      background: #3866FF;
      padding: 20px 30px;
      border-radius: 25px;
      color: white;
      font-size: 28px;
      font-weight: 900;
      text-align: center;
      z-index: 5;
      box-shadow: 0px 6px 18px rgba(0, 0, 0, 0.45);
    }

  </style>
</head>
```

```
<body>

  <video id="camera" autoplay playsinline></video>

  <div id="totalBox">$0 USD acumulado</div>

  <button id="mainButton">
    <p id="mainButtonText">Escanear Billete</p>
    <p id="subText">Click para escanear. Mantén presionado para escuchar el total.</p>
  </button>

  <div id="resultBox">
    <p id="resultText">$0 Dollar USD DETECTADO</p>
    <p id="subText">Click para continuar</p>
  </div>

  <script>
    function speak(text) {
      const msg = new SpeechSynthesisUtterance(text);
      msg.lang = "es-ES";
      window.speechSynthesis.speak(msg);
    }

    window.onload = () => {
      speak("Bienvenido. Para obtener una detección precisa, coloque el billete frente a la cámara, completamente extendido y sobre una superficie plana. Asegúrese de que haya buena iluminación. Si el billete se ve oscuro, acérquelo a una zona más luminada. Cuando esté listo, toque el botón para escanear. Mantenga presionado el botón para escuchar el total acumulado.");
    };

    async function startCamera() {
      try {
        const stream = await navigator.mediaDevices.getUserMedia({ video: true });
        camera.srcObject = stream;
      } catch {
        speak("No se pudo activar la cámara.");
      }
      startCamera();
    }

    let isLongPress = false;
    let longPressTimer;
    let total = 0;

    async function scanBill() {
      speak("Escaneeando, espere un momento.");

      const video = document.getElementById("camera");
      const canvas = document.createElement("canvas");
      canvas.width = video.videoWidth;
      canvas.height = video.videoHeight;
      canvas.getContext("2d").drawImage(video, 0, 0);

      const blob = await new Promise(resolve => canvas.toBlob(resolve, "image/jpeg"));
      const formData = new FormData();
      formData.append("image", blob);

      const res = await fetch("/predict", { method: "POST", body: formData });
      const data = await res.json();

      showResult(data.class, data.confidence);
    }

    function showResult(amount, confidence) {
      const confPercent = Math.round(confidence * 100);

      const val = parseInt(amount);
      if (!isNaN(val)) {
        total += val;
        document.getElementById("totalBox").textContent = "$" + total + " USD acumulado";
        speak("Total acumulado: " + total + " dólares.");
      }

      resultText.innerHTML =
        "$" + amount + " Dollar USD DETECTADO<br><span style='font-size:22px;'>Confianza: " +
        confPercent + "%</span>";

      resultBox.style.display = "block";
      mainButton.style.display = "none";

      speak("Detectado billete de " + amount + " dólares con una confianza del " + confPercent +
        " por ciento.");
    }

    function resetToDetection() {
      resultBox.style.display = "none";
      mainButton.style.display = "block";
      speak("Listo. Puede escanear otro billete.");
    }

    mainButton.addEventListener("mousedown", () => {
      longPressTimer = setTimeout(() => {
        isLongPress = true;
        speak("El total acumulado es " + total + " dólares.");
      }, 600);
    });

    mainButton.addEventListener("mouseup", () => {
      clearTimeout(longPressTimer);

      if (!isLongPress) {
        scanBill();
      }

      isLongPress = false;
    });

    mainButton.addEventListener("mouseleave", () => {
      clearTimeout(longPressTimer);
    });

    resultBox.onclick = resetToDetection;
  </script>

</body>

</html>
```

```

import tensorflow as tf
import os

# --- Ruta del dataset ---
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
base_dir = r"C:\9mo-Semestre\UX\datasets\usd\USA currency"

# --- Configuración ---
IMAGE_SIZE = 224
BATCH_SIZE = 32

# --- Data Augmentation ---
datagen = tf.keras.preprocessing.image.ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    zoom_range=0.2,
    brightness_range=[0.3, 1.5],
    shear_range=0.2,
    fill_mode="nearest"
)

train_generator = datagen.flow_from_directory(
    base_dir,
    target_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
    subset="training"
)

val_generator = datagen.flow_from_directory(
    base_dir,
    target_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
    subset="validation"
)

labels = '\n'.join(sorted(train_generator.class_indices.keys()))
with open(os.path.join(BASE_DIR, 'labels.txt'), 'w') as f:
    f.write(labels)

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(IMAGE_SIZE, IMAGE_SIZE, 3),
    include_top=False,
    weights='imagenet'
)
base_model.trainable = False

num_classes = train_generator.num_classes

model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.Conv2D(32, 3, activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    train_generator,
    epochs=10,
    validation_data=val_generator
)

base_model.trainable = True

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_fine = model.fit(
    train_generator,
    epochs=15,
    validation_data=val_generator
)

model.save(os.path.join(BASE_DIR, "usd_model.h5"))

```

Clave	Fecha	Versión	Elaboró	Revisó
APP-SCAN-001	12/1/2025	1.0	Dante Alegria	Andrea Margarita Balandran Felix

Firma del líder del proyecto